RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 3a: Model Planning and Building - Classification

Roll Number: 241801187 **Date:** 06-10-2026

Dataset: Telco Customer Churn (IBM) **Tools:** Python, Pandas, Matplotlib, Seaborn, scikit-learn

Learning Objective

To plan, build, and evaluate basic machine learning models using real-world datasets.

Problem Statement

A telecommunications company has noticed that a growing number of customers are discontinuing its services, resulting in reduced revenue and increased customer acquisition costs. After data preprocessing and exploratory analysis, the company now aims to develop predictive models that identify customers likely to churn. As a Data Scientist, the task is to build and compare multiple classification models, evaluate them with appropriate metrics, identify the most effective model, and interpret the important factors influencing churn.

Task 1: Load CustomerChurn_Preprocessed.csv created in Ex 2

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, accuracy_score,
                             precision_score, recall_score, f1_score)
sns.set_theme(style='whitegrid')

df = pd.read_csv('CustomerChurn_Preprocessed.csv')
print("Shape:", df.shape)
print("Missing values:", df.isnull().sum().sum())
print("Churn rate (%):", round(df['Churn'].mean() * 100, 2))
```

Out [1]:

```
Shape: (7043, 39)
Missing values: 0
Churn rate (%): 26.54
```

Task 2: Split the dataset into Training (80%) and Testing (20%)

In [2]:

```
X = df.drop(columns='Churn')
y = df['Churn']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("Churn rate - train: %.2f%% | test: %.2f%%" % (y_train.mean()*100, y_test.mean()*100))
```

Out [2]:

```
X_train: (5634, 38) | X_test: (1409, 38)
Churn rate - train: 26.54% | test: 26.54%
```

> **Observation:** The data is split 80:20 into 5,634 training and 1,409 testing rows. Stratifying on Churn keeps the churn rate at 26.54% in both sets, and random_state=42 makes the split reproducible.

Model Building

Tasks 3-5: Logistic Regression, Decision Tree and Random Forest

In [3]:

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=200, random_state=42),
}
preds = {}
for name, model in models.items():
    model.fit(X_train, y_train)          # train
    preds[name] = model.predict(X_test)  # predict churn on the test set
    print(f"{name}: trained and predicted {len(preds[name])} test customers")
```

Out [3]:

```
Logistic Regression: trained and predicted 1409 test customers
Decision Tree: trained and predicted 1409 test customers
Random Forest: trained and predicted 1409 test customers
```

> **Observation:** Logistic Regression is a linear baseline. The Decision Tree is limited to max_depth=5 to reduce overfitting. The Random Forest averages 200 trees. All three predict churn (0 = No, 1 = Yes) for the same 1,409 unseen customers.

Model Evaluation

Task 6: Confusion matrix, Accuracy, Precision, Recall and F1-Score

In [4]:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
rows = []
for ax, (name, p) in zip(axes, preds.items()):
    cm = confusion_matrix(y_test, p)
    ConfusionMatrixDisplay(cm, display_labels=['No', 'Yes']).plot(ax=ax, cmap='Blues', colorbar=False)
    ax.set_title(name)
    print(f"\n{name} confusion matrix:\n{cm}")
    rows.append([name, accuracy_score(y_test, p), precision_score(y_test, p),
                 recall_score(y_test, p), f1_score(y_test, p)])
plt.tight_layout(); plt.show()

results = pd.DataFrame(rows, columns=['Model','Accuracy','Precision','Recall','F1-Score']).set_index('Model')
print(results.round(4))
```

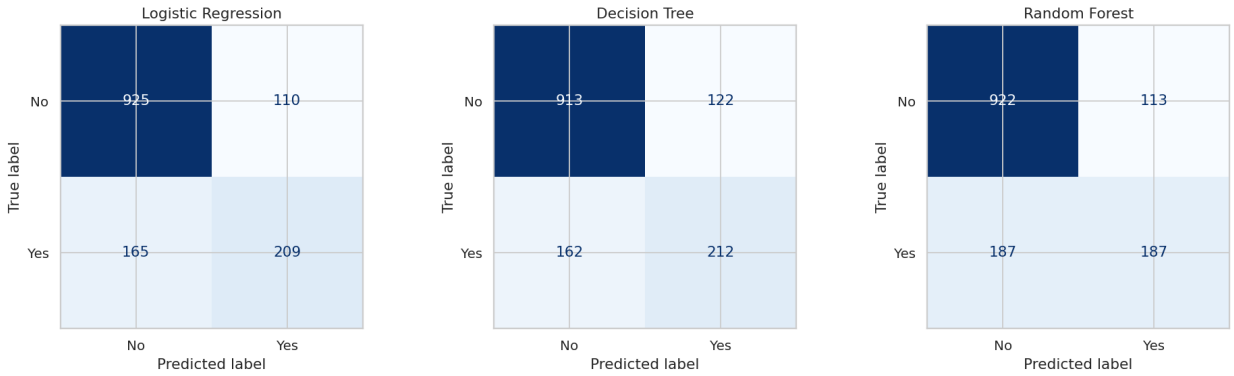
Out [4]:

```
Logistic Regression confusion matrix:
[[ 925  110]
 [ 165  209]]

Decision Tree confusion matrix:
[[ 913  122]
 [ 162  212]]

Random Forest confusion matrix:
[[ 922  113]
 [ 187  187]]

                     Accuracy  Precision  Recall  F1-Score
Logistic Regression    0.8048     0.6552  0.5588    0.6032
Decision Tree          0.7984     0.6347  0.5668    0.5989
Random Forest          0.7871     0.6233  0.5000    0.5549
```



> **Observation:** Logistic Regression correctly finds 209 of the 374 churners (recall 55.88%) with 110 false alarms. The Decision Tree finds 212 with 122 false alarms. The Random Forest finds only 187 and misses 187. All three models predict the majority class (No) well, but recall on churners stays near 50-57% because the data is imbalanced.

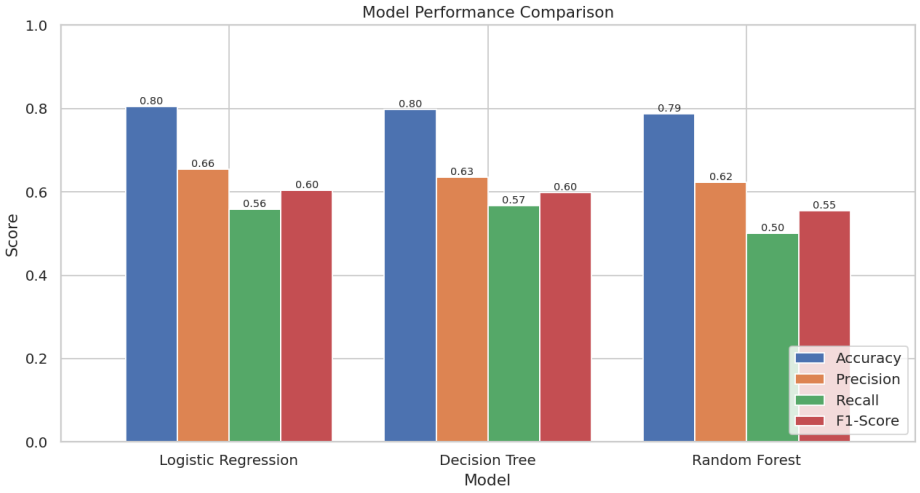
Task 7: Compare all three models in a single bar chart

In [5]:

```
ax = results.plot(kind='bar', figsize=(10, 5.5), rot=0, width=0.8)
for c in ax.containers:
    ax.bar_label(c, fmt='%.2f', fontsize=8)
plt.title('Model Performance Comparison')
plt.ylabel('Score'); plt.ylim(0, 1)
plt.legend(loc='lower right')
plt.tight_layout(); plt.show()
```

Out [5]:

(grouped bar chart shown below)



Task 8: Identify the best-performing model

In [6]:

```
best = results['F1-Score'].idxmax()   # F1 balances precision and recall on imbalanced data
print("Best model by F1-Score:", best)
print(results.loc[best].round(4).to_string())
```

Out [6]:

```
Best model by F1-Score: Logistic Regression
Accuracy     0.8048
Precision    0.6552
Recall       0.5588
F1-Score     0.6032
```

> **Observation:** Logistic Regression is the best model: highest accuracy (80.48%), precision (65.52%) and F1-Score (60.32%). The Decision Tree has a slightly higher recall (56.68% vs 55.88%) but lower F1. The Random Forest is weakest here (F1 55.49%). The margins between models are small, so on this dataset a simple, interpretable model performs as well as the more complex ones.

Business Recommendation

Task 9: Drivers of churn from the best model

In [7]:

```
coef = pd.Series(models['Logistic Regression'].coef_[0], index=X.columns).sort_values()
print("Strongest factors REDUCING churn:\n", coef.head(5).round(3))
print("\nStrongest factors INCREASING churn:\n", coef.tail(5).round(3))
```

Out [7]:

```
Strongest factors REDUCING churn:
 tenure                            -1.228
Contract                           -0.693
InternetService_DSL                -0.665
MonthlyCharges                     -0.579
OnlineBackup_No internet service   -0.315

Strongest factors INCREASING churn:
 StreamingMovies_Yes                0.183
PaymentMethod_Electronic check      0.185
PaperlessBilling                    0.371
TotalCharges                        0.503
InternetService_Fiber optic         0.616
```

> **Observation:** Because the features are standardised, coefficient size shows relative influence. Longer tenure and longer contracts lower churn the most, while fiber optic internet, paperless billing and electronic check payment raise it. TotalCharges and MonthlyCharges are strongly correlated with tenure and each other (see Ex 2 heatmap), so their individual signs should be read with caution.

Recommendations to reduce churn

1. **Move month-to-month customers onto longer contracts.** Contract and tenure are the strongest churn reducers in the model, and Ex 2 showed month-to-month churn of 42.71% against 2.83% for two-year plans. Offer discounts, free upgrades or price locks for 12- and 24-month commitments, especially in the first year when churn is 47.44%.
2. **Run a retention programme for fiber optic customers.** Fiber optic is the strongest positive churn driver in the model (churn 41.89% against 18.96% for DSL). Investigate service quality and pricing for this plan, and bundle online security, tech support and backup, since customers without those add-ons rank among the top Random Forest features.
3. **Move customers to automatic payment and target early-life customers.** Electronic check users are the most common payment method among churners (1,071 of 1,869). Offer a small incentive to switch to bank transfer or credit card auto-pay, and use the model to flag new, high-risk customers for onboarding calls and early check-ins.

Result

Thus three classification models were built on the preprocessed Telco Customer Churn dataset using an 80:20 stratified split. Logistic Regression, Decision Tree and Random Forest were evaluated with confusion matrices, accuracy, precision, recall and F1-Score, and compared in a single bar chart. Logistic Regression performed best (accuracy 80.48%, F1-Score 60.32%). Contract length, tenure, fiber optic internet and payment method were identified as key churn factors, and three business recommendations were made to reduce churn.